

INSTITUTE OF MATHEMATICS & COMPUTER SCINENCE (IMCS)
HUMAN-COMPUTER INTERACTION (HCI) & COMPUTER GRAPHICS (CG)
HCI&CG Assignment

LAB 1:

Display Density Metrics & Image Array Mechanics in NumPy

Submitted By:

Name: Sanauallah Odho

Roll No: 2k24\CSE\131

Semester: 6TH Semster

Section: *Morning CSE*

Submitted To:

Teacher Name: Rajesh Kumar

Date: 12-09-2k26

lab	01
language	Python
libraries	NumPy, Pillow, Matplotlib
Input images	sample.jpg

TASK 1: DISPLAY PIXEL DENSITY (PPI/DPI) CALCULATOR:

Objective:

To calculate total pixel count, aspect ratio, DPI/PPI, and display density category for a screen.

Python Source Code:

```
import math

# =====
# TASK 1: DISPLAY METRICS
# =====

# Getting screen resolution and size from the user
Wpx = int(input("Enter horizontal resolution (pixels): "))
Hpx = int(input("Enter vertical resolution (pixels): "))
Dinches = float(input("Enter physical diagonal size (inches): "))

# Calculate total number of pixels
total_pixels = Wpx * Hpx

# Find greatest common divisor for aspect ratio
g = math.gcd(Wpx, Hpx)

# Simplify the aspect ratio
aspect_ratio = f"{Wpx // g}:{Hpx // g}"

# Calculate diagonal pixels
diagonal_pixels = math.sqrt(Wpx**2 + Hpx**2)

# Calculate DPI / PPI
dpi = diagonal_pixels / Dinches

# Classify display density
if dpi < 100:
    density = "Low Density (Standard Monitor)"

elif dpi <= 200:
    density = "Medium Density (HD Display)"

else:
```

```

    density = "High Density (Retina / Mobile)"

# Display the calculated results
print("\n--- DISPLAY METRICS ANALYSIS ---")

print(f"Total Pixel Count : {total_pixels:,} pixels")

print(f"Aspect Ratio : {aspect_ratio}")

print(f"Calculated DPI : {dpi:.2f} DPI")

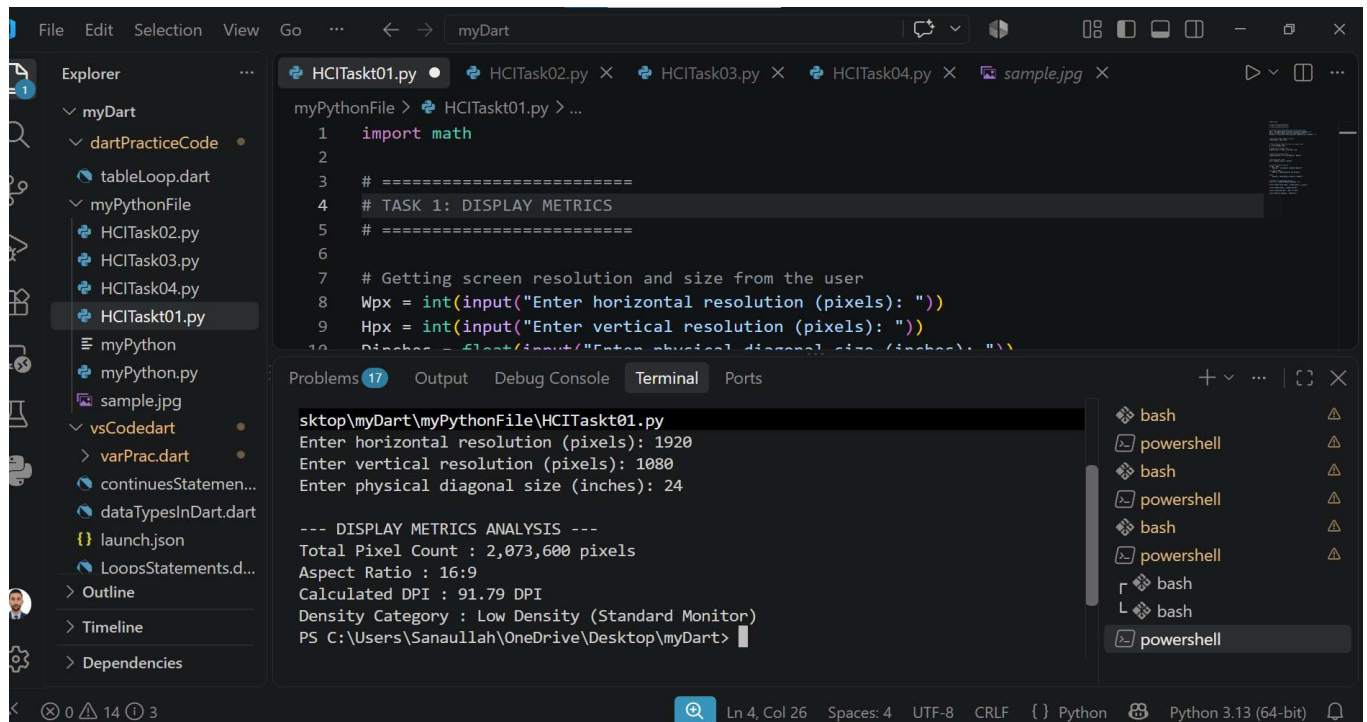
print(f"Density Category : {density}")

```

Task01:

Input: 1920 × 1080 pixels, 24 inches.

Task01 Output:



```

myPythonFile > HCITask01.py > ...
1  import math
2
3  # =====
4  # TASK 1: DISPLAY METRICS
5  # =====
6
7  # Getting screen resolution and size from the user
8  Wpx = int(input("Enter horizontal resolution (pixels): "))
9  Hpx = int(input("Enter vertical resolution (pixels): "))
10 Diagonal = float(input("Enter physical diagonal size (inches): "))

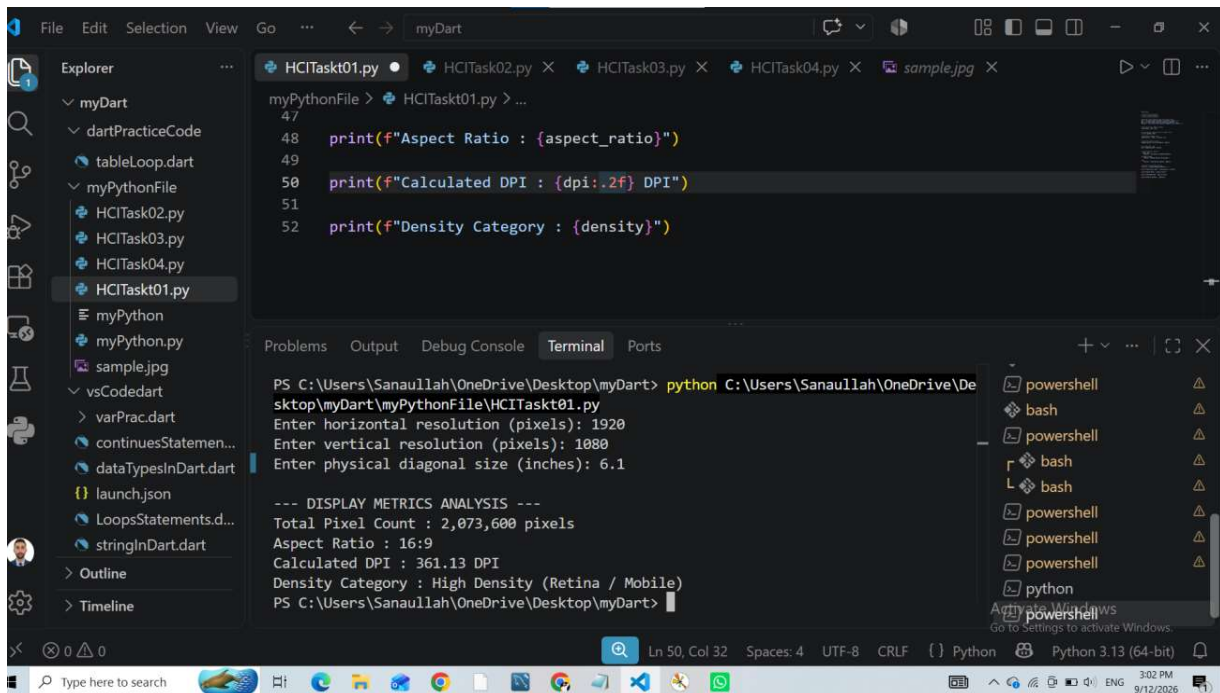
sktop\myDart\myPythonFile\HCITask01.py
Enter horizontal resolution (pixels): 1920
Enter vertical resolution (pixels): 1080
Enter physical diagonal size (inches): 24

--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count : 2,073,600 pixels
Aspect Ratio : 16:9
Calculated DPI : 91.79 DPI
Density Category : Low Density (Standard Monitor)
PS C:\Users\Sanaullah\OneDrive\Desktop\myDart>

```

Output: using mobile Screen input

Input: 1920×1080 pixels, 6.1 inches.



The screenshot shows a VS Code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a project named 'myDart' with several files. The terminal shows the execution of a Python script 'HCITask01.py' which prompts the user to enter horizontal resolution (1920), vertical resolution (1080), and physical diagonal size (6.1). The script then displays metrics: Total Pixel Count (2,073,600), Aspect Ratio (16:9), Calculated DPI (361.13), and Density Category (High Density (Retina / Mobile)).

```
myPythonFile > HCITask01.py > ...
47
48 print(f"Aspect Ratio : {aspect_ratio}")
49
50 print(f"Calculated DPI : {dpi:.2f} DPI")
51
52 print(f"Density Category : {density}")

PS C:\Users\Sanaullah\OneDrive\Desktop\myDart> python C:\Users\Sanaullah\OneDrive\Desktop\myDart\myPythonFile\HCITask01.py
Enter horizontal resolution (pixels): 1920
Enter vertical resolution (pixels): 1080
Enter physical diagonal size (inches): 6.1

--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count : 2,073,600 pixels
Aspect Ratio : 16:9
Calculated DPI : 361.13 DPI
Density Category : High Density (Retina / Mobile)
PS C:\Users\Sanaullah\OneDrive\Desktop\myDart>
```

Task02

TASK 2: ENVIRONMENT SETUP & SYNTHETIC IMAGE MATRIX CREATION

Objective:

To create a $300 \times 400 \times 3$ synthetic RGB image array using NumPy and inspect its properties.

Required Packages:

`pip install numpy pillow matplotlib`

Source Code:

```
import numpy as np
import matplotlib.pyplot as plt
```

```

from PIL import Image

# =====
# TASK 2: SYNTHETIC IMAGE
# =====

# Create a 300 x 400 RGB image filled with zeros
img = np.zeros((300, 400, 3), dtype=np.uint8)

# Fill each quadrant with a different color

# Top-left quadrant = Red
img[:150, :200] = [255, 0, 0]

# Top-right quadrant = Green
img[:150, 200:] = [0, 255, 0]

# Bottom-left quadrant = Blue
img[150:, :200] = [0, 0, 255]

# Bottom-right quadrant = White
img[150:, 200:] = [255, 255, 255]

# Print image array information
print("\n--- SYNTHETIC MATRIX METRICS ---")

# Print image height, width, and channels
print("Array Shape (H, W, C) :", img.shape)

# Print the data type of pixels
print("Data Type           :", img.dtype)

# Print total number of values in the array
print("Total Elements       :", img.size, "values")

# Print memory used by the image
print(
    "Memory Footprint      :",

```

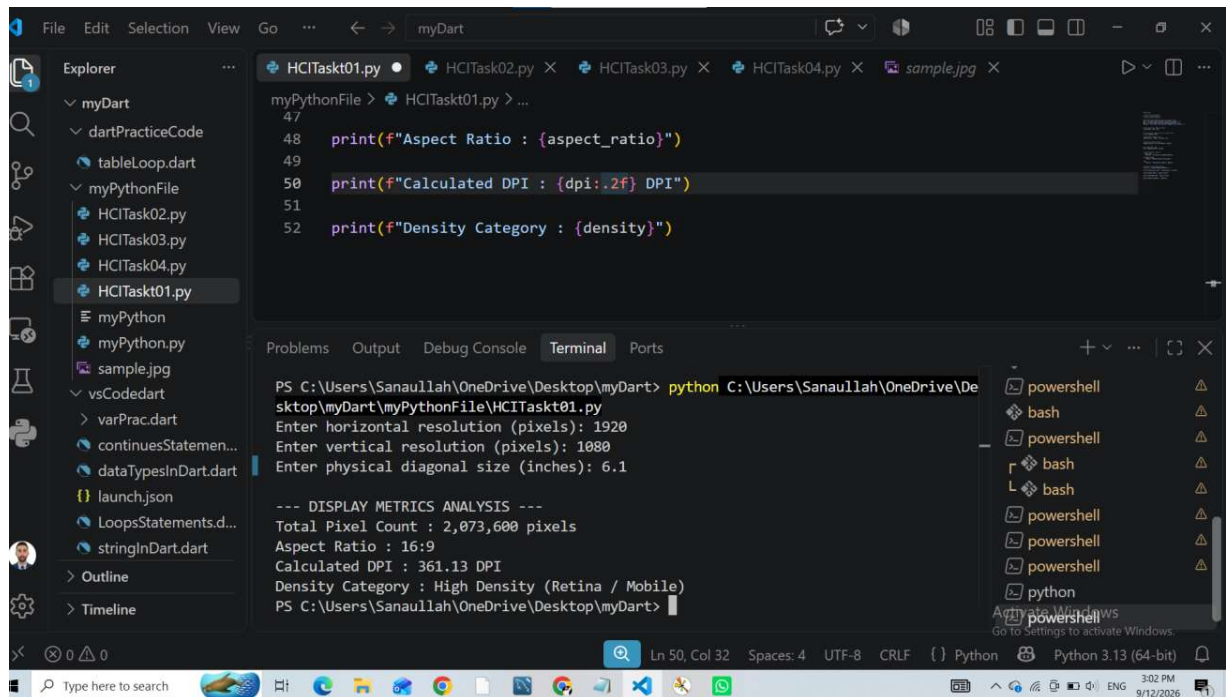
```
    img.nbytes,  
    "bytes",  
    f"({img.nbytes / 1024:.2f} KB)"  
)  
  
# =====  
# SHOW IMAGE  
# =====  
  
# Display the generated image  
plt.imshow(img)  
  
# Hide the axes  
plt.axis("off")  
  
# Show the image  
plt.show()  
  
# Save the generated image as a PNG file  
Image.fromarray(img).save("synthetic_image.png")
```

In the program output showing: Array Shape (H, W, C)

Data Type

Total Elements

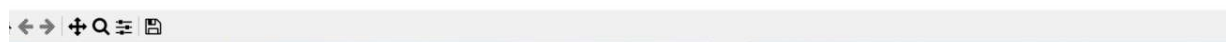
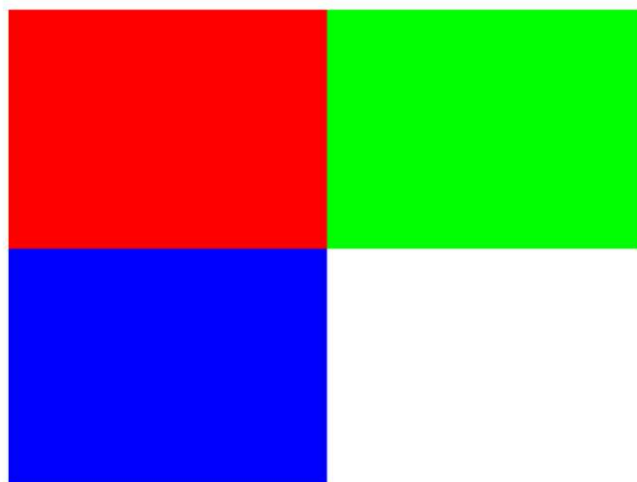
Memory Footprint



four-quadrant images:

RED GREEN
BLUE WHITE

Figure 1



TASK 3: CHANNEL SLICING & ISOLATION

Objective:

To load an image, extract Red, Green, and Blue channels, and display individual channel results.

I used this Input Image:

Input File: sample.jpg



Python Source Code:

```
# =====  
# TASK 3: CHANNEL SLICING  
# =====  
  
import numpy as np  
import matplotlib.pyplot as plt  
from PIL import Image  
import os  
  
# Find the folder where this Python file is saved  
script_folder = os.path.dirname(os.path.abspath(__file__))
```

```
# Set the path of the input image
input_image = os.path.join(script_folder, "sample.jpg")

# Check if the image file exists
if not os.path.exists(input_image):
    raise FileNotFoundError(
        f"Image file not found. Please put sample.jpg in: \n{script_folder}"
    )

# Load the image and convert it to RGB format
image = np.array(Image.open(input_image).convert("RGB"))

# Extract Red, Green, and Blue channels
# Channel 0 = Red
# Channel 1 = Green
# Channel 2 = Blue

red = image[:, :, 0]
green = image[:, :, 1]
blue = image[:, :, 2]

# Create empty images with the same shape as the original image
red_only = np.zeros_like(image)
green_only = np.zeros_like(image)
blue_only = np.zeros_like(image)

# Add only the Red channel
red_only[:, :, 0] = red

# Add only the Green channel
green_only[:, :, 1] = green

# Add only the Blue channel
blue_only[:, :, 2] = blue

# Print image and channel information
```

```
print("\n--- CHANNEL EXTRACTION SUMMARY ---")

# Print the original image shape
print("Original Image Shape :", image.shape)

# Print Red channel shape and mean intensity
print(
    f"Red Channel 2D Shape   : {red.shape} | "
    f"Mean Intensity: {red.mean():.2f}"
)

# Print Green channel shape and mean intensity
print(
    f"Green Channel 2D Shape : {green.shape} | "
    f"Mean Intensity: {green.mean():.2f}"
)

# Print Blue channel shape and mean intensity
print(
    f"Blue Channel 2D Shape  : {blue.shape} | "
    f"Mean Intensity: {blue.mean():.2f}"
)

# Create a figure for displaying the results
plt.figure(figsize=(12, 7))

# ----- TOP ROW: COLOR CHANNELS -----

# Display Red-only image
plt.subplot(2, 3, 1)
plt.imshow(red_only)
plt.title("Red-Only")
plt.axis("off")

# Display Green-only image
plt.subplot(2, 3, 2)
plt.imshow(green_only)
plt.title("Green-Only")
plt.axis("off")
```

```
# Display Blue-only image
plt.subplot(2, 3, 3)
plt.imshow(blue_only)
plt.title("Blue-Only")
plt.axis("off")

# ----- BOTTOM ROW: GRAYSCALE INTENSITY -----

# Display Red channel as grayscale
plt.subplot(2, 3, 4)
plt.imshow(red, cmap="gray")
plt.title("Red Intensity")
plt.axis("off")

# Display Green channel as grayscale
plt.subplot(2, 3, 5)
plt.imshow(green, cmap="gray")
plt.title("Green Intensity")
plt.axis("off")

# Display Blue channel as grayscale
plt.subplot(2, 3, 6)
plt.imshow(blue, cmap="gray")
plt.title("Blue Intensity")
plt.axis("off")

# Adjust the spacing between plots
plt.tight_layout()

# Show all six results
plt.show()
```

Output03task:

```
myPythonFile > HCITask03.py > ...
113 plt.imshow(green, cmap="gray")
114 plt.title("Green Intensity")
115 plt.axis("off")
116
117 # Display Blue channel as grayscale
118 plt.subplot(2, 3, 6)
119 plt.imshow(blue, cmap="gray")
120 plt.title("Blue Intensity")
121 plt.axis("off")
122
```

Problems Output Debug Console **Terminal** Ports

```
PS C:\Users\Sanaullah\OneDrive\Desktop\myDart> python C:\Users\Sanaullah\OneDrive\Desktop\myDart\myPythonFile\HCITask03.py

--- CHANNEL EXTRACTION SUMMARY ---
Original Image Shape : (365, 547, 3)
Red Channel 2D Shape : (365, 547) | Mean Intensity: 119.26
Green Channel 2D Shape : (365, 547) | Mean Intensity: 93.60
Blue Channel 2D Shape : (365, 547) | Mean Intensity: 99.32
```

2 × 3 Matplotlib figure.

It contains:

Red-Only Green-Only Blue-Only

Red Intensity Green Intensity Blue Intensity



Task04

TASK 4: SPATIAL DOWNSAMPLING & PIXELATION VIA STRIDING

Objective:

To down sample an image using NumPy striding with $N = 8$ and observe the pixelation effect after re-expansion.

```
# =====  
# TASK 4: DOWNSAMPLING & STRIDING  
# =====  
  
import numpy as np  
import matplotlib.pyplot as plt  
from PIL import Image  
import os  
  
# Find the folder of this Python file  
script_folder = os.path.dirname(os.path.abspath(__file__))  
  
# Set image path  
input_image = os.path.join(script_folder, "sample.jpg")
```

```
# Check if image exists
if not os.path.exists(input_image):
    raise FileNotFoundError(
        f"Image file not found. Please put sample.jpg in: \n{script_folder}"
    )

# Load image and convert to RGB
image = np.array(Image.open(input_image).convert("RGB"))

# Set downsampling factor
N = 8

# Take every 8th row and column
downsampled = image[::N, ::N, :]

# Expand image again using repeated pixels
reexpanded = np.repeat(
    np.repeat(downsampled, N, axis=0),
    N,
    axis=1
)

# Get original image dimensions
original_height, original_width, channels = image.shape

# Get downsampled dimensions
down_height, down_width, down_channels = downsampled.shape

# Calculate height reduction percentage
height_reduction = (
    (original_height - down_height)
    / original_height
) * 100

# Calculate width reduction percentage
width_reduction = (
    (original_width - down_width)
```

```

    / original_width
) * 100

# Get memory used by both images
original_memory = image.nbytes
downsampled_memory = downsampled.nbytes

# Calculate memory reduction percentage
memory_reduction = (
    (original_memory - downsampled_memory)
    / original_memory
) * 100

# Print results
print("\n--- DOWNSAMPLING SUMMARY ---")

print("Original Image Shape      :", image.shape)

print("Downsampled Image Shape   :", downsampled.shape)

print("Re-expanded Image Shape    :", reexpanded.shape)

print(f"Height Reduction           : {height_reduction:.2f}%")

print(f"Width Reduction            : {width_reduction:.2f}%")

print(f"Original Memory             : {original_memory:,} bytes")

print(f"Downsampled Memory          : {downsampled_memory:,} bytes")

print(f"Memory Reduction            : {memory_reduction:.2f}%")

# Create figure for three images
plt.figure(figsize=(15, 5))

# Show original image
plt.subplot(1, 3, 1)

```

```
plt.imshow(image)
plt.title("Original Image")
plt.axis("off")

# Show downsampled image
plt.subplot(1, 3, 2)
plt.imshow(downsampled)
plt.title("Downsampled (N=8)")
plt.axis("off")

# Show re-expanded image
plt.subplot(1, 3, 3)
plt.imshow(reexpanded)
plt.title("Re-expanded Image")
plt.axis("off")

# Adjust spacing
plt.tight_layout()

# Display the figure
plt.show()
```

Output04task:

Output

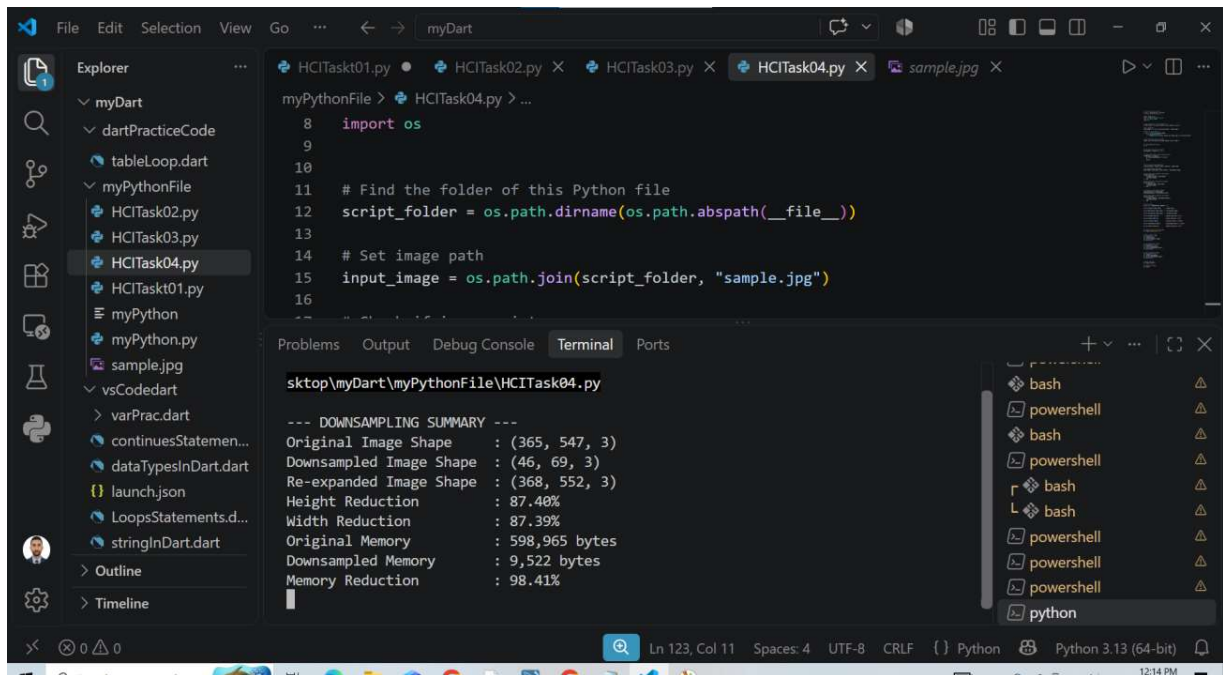
Original Shape

Downsampled Shape

Re-expanded Shape

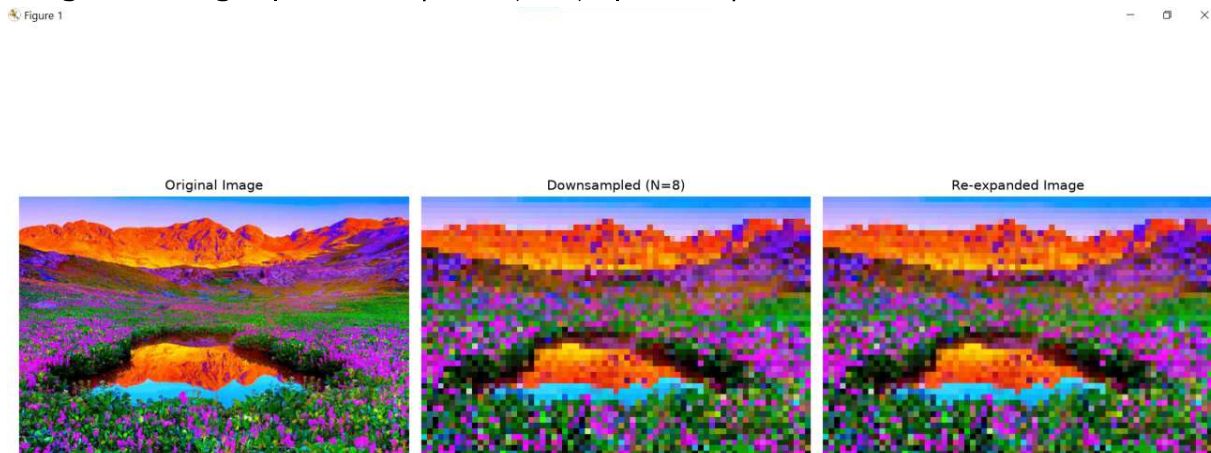
Dimension Reduction

Memory Savings



Matplotlib figure:

Original Image | Downsampled (N=8) | Re-expanded / Pixelated



Final Question ANSWER:

Q: Why does adding an Alpha channel (RGBA) increase an image array's memory consumption by 33% compared to standard RGB?

Answer:

RGB uses three color channels: Red, Green, and Blue. RGBA adds a fourth channel called Alpha, which stores transparency information. When each channel uses 8-bit (1-byte) storage, RGB requires 3 bytes per pixel while RGBA requires 4 bytes per pixel. Therefore, the increase is:

$$(4 - 3) / 3 \times 100 = 33.33\%$$

Hence, RGBA uses approximately **33% more memory** than RGB.

Final Report Order:

1. cover Page.
2. Task 1
 - |— Objective
 - |— Python Code
 - |— Desktop Output
 - |— Smartphone Output
3. Task 2
 - |— Objective
 - |— Required Packages
 - |— Python Code
 - |— Output + Image
4. Task 3
 - |— Objective
 - |— Input Image
 - |— Python Code
 - |— 2×3 Output Figure
5. Task 4

- |— Objective
- |— Python Code
- |— Terminal Output
- |— 3-image Output Figure

6.Final Question Answer

